RATAN TATA MAHARASHTRA STATE SKILLS UNIVERSITY

School of Science Engineering and Technology Skills

Department of Engineering

Subject – AWS Lab

Experiment – 8

Title: Monitoring and Auditing in AWS

Introduction:

Visibility is the cornerstone of cloud security and operational excellence. This lab focuses on two primary services:

- AWS CloudTrail: Records API calls and user activity across your AWS infrastructure (The "Who, What, and When").
- Amazon CloudWatch: Monitors resource performance, collects logs, and triggers automated responses via alarms (The "How is it performing?").

Lab Objective:

1. Analyze CloudTrail event history to audit account activity.
2. Configure a CloudWatch Alarm to monitor specific resource metrics.
3. Design a CloudWatch Dashboard to visualize real-time performance data.
4. Integrate CloudWatch Logs with metric filters for proactive alerting.

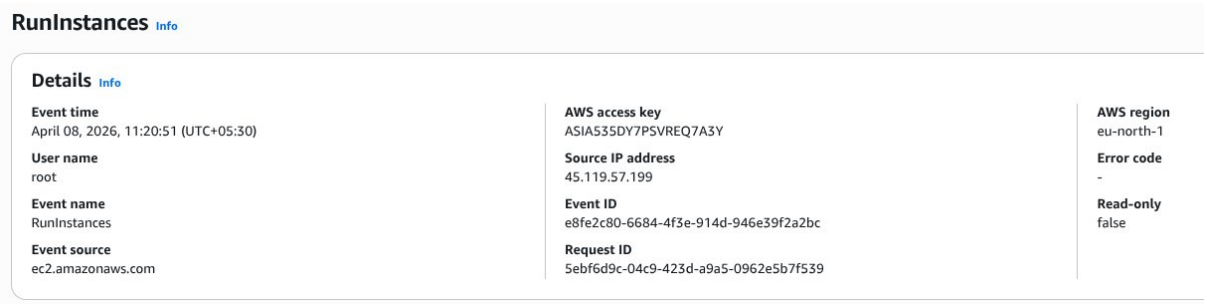
Part 1: Exploring AWS CloudTrail

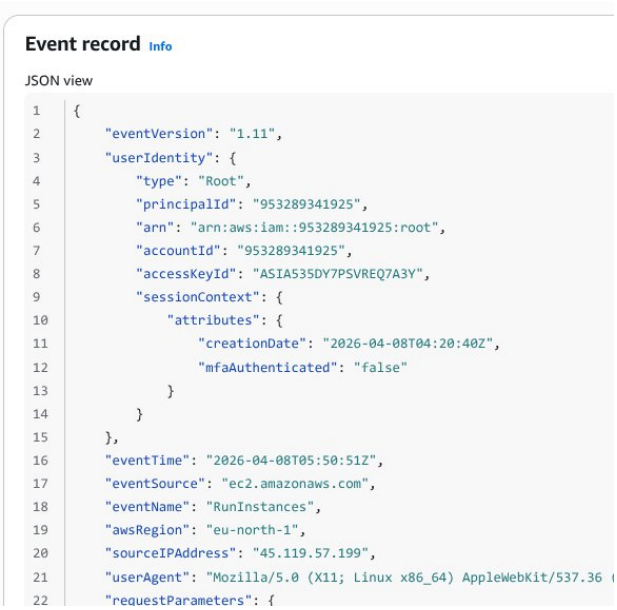
CloudTrail is enabled by default the moment you create an AWS account. It provides a history of the last 90 days of management events.

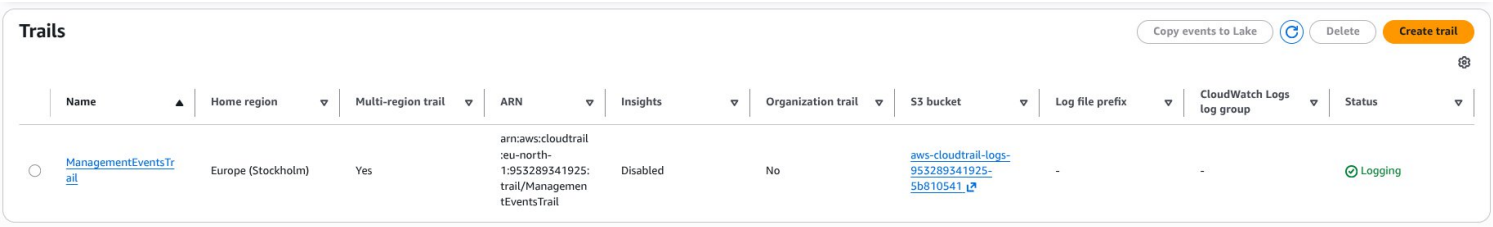
Step-by-Step Instructions:

1. **Navigate to CloudTrail:** Search for "CloudTrail" in the AWS Management Console.
2. **View Event History:** Click on **Event history** in the left navigation pane.
3. **Filter Events:** Change the filter to **Event name** and type RunInstances. This shows who launched an EC2 instance and when.
    - Click on an event to view the **JSON record**.

4. **Create a Trail (Optional but recommended):**
   - Click **Create trail**.
   - Provide a name (e.g., ManagementEventsTrail).
   - Choose an S3 bucket to store logs permanently (beyond the 90-day default).





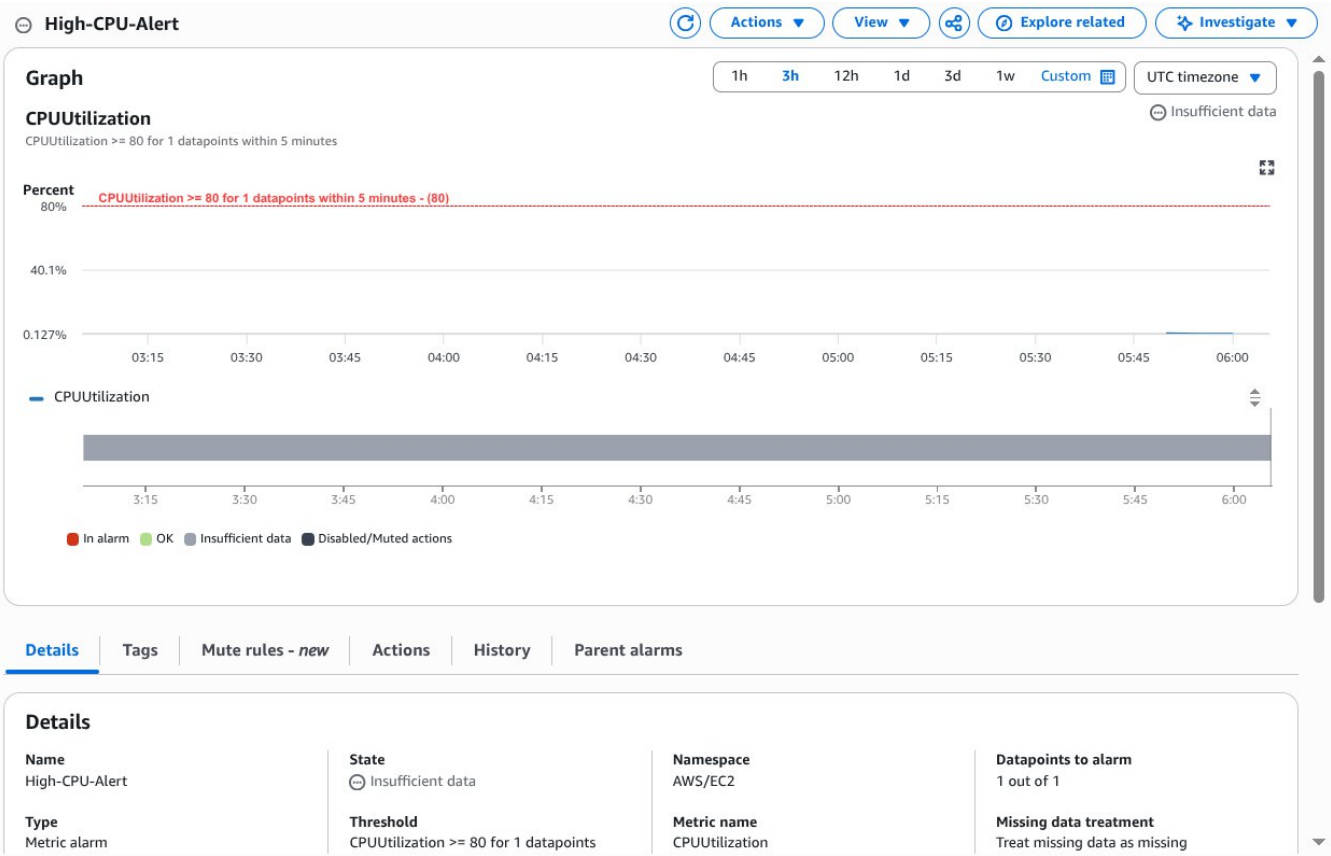


Part 2: Amazon CloudWatch Metrics & Alarms

Metrics are variables about your resources (e.g., CPU Utilization). Alarms watch a single metric and perform actions based on thresholds.

Step-by-Step Instructions:

1. **Select a Metric:** Go to the **CloudWatch** console.
    - Select **All metrics** > **EC2** > **Per-Instance Metrics**.
    - Select CPUUtilization for an active EC2 instance.
2. **Create an Alarm:**
    - Click the **Graphed metrics** tab and select the bell icon (Create Alarm).
    - **Conditions:** Set the threshold to **Static**. Define: *Whenever CPUUtilization is... Greater/Equal than... 80%.*
    - **Actions:** Configure an **SNS Topic** to send an email notification when the alarm state is "In Alarm".
3. **Review and Create:** Give the alarm a name (e.g., High-CPU-Alert) and save.



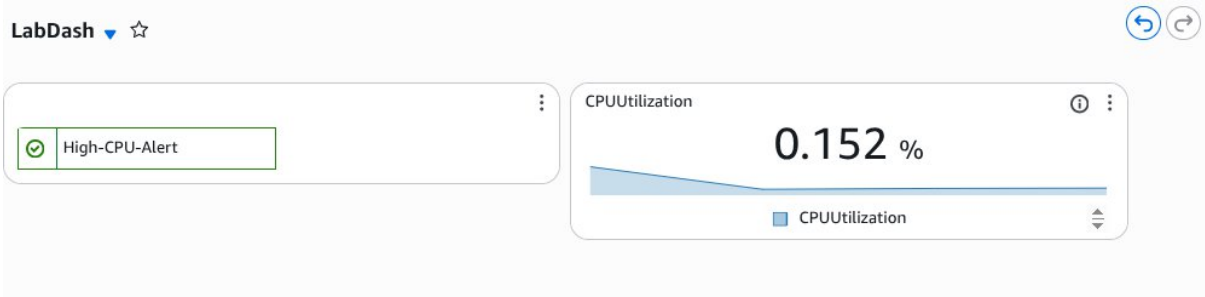
Part 3: Building a CloudWatch Dashboard

Dashboards provide a single-pane-of-glass view of your infrastructure.

Step-by-Step Instructions:

1. **Navigate to Dashboards:** In the CloudWatch sidebar, select **Dashboards** > **Create dashboard**.

2. **Add Widgets:**
    - Choose **Line** or **Stacked area** for time-series data like CPU or Network In/Out.
    - Choose **Number** for a quick glance at the current status of an alarm.
3. **Customization:** Drag and resize widgets to organize your view. Click **Save dashboard**.

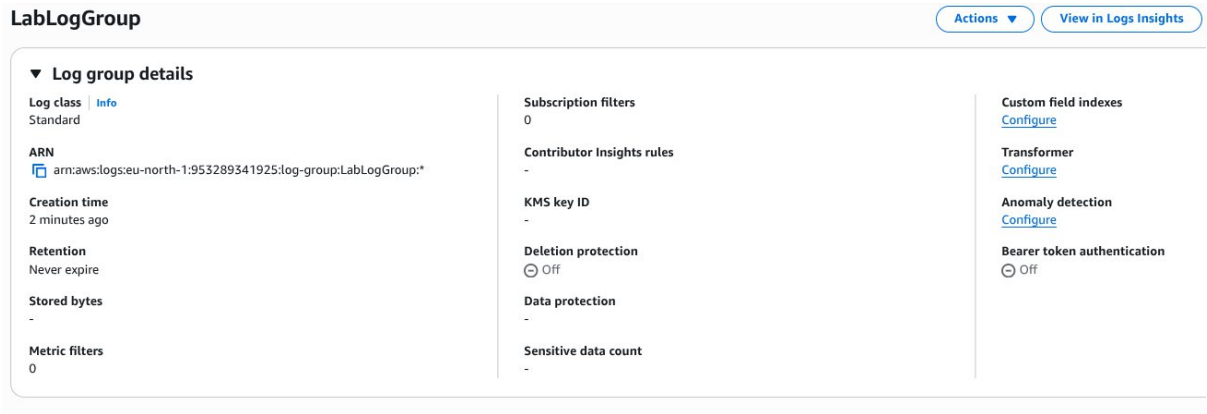


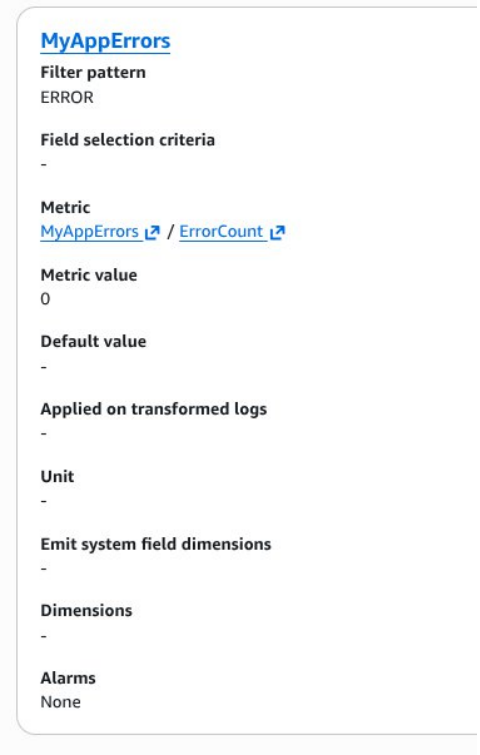
Part 4: CloudWatch Logs and Metric Filters

Logs allow you to monitor application-specific behavior by searching for strings within log files.

Step-by-Step Instructions:

1. **Log Groups:** Navigate to **Logs** > **Log groups**.
2. **Metric Filter:** Select a log group (e.g., from a Lambda function or EC2) and go to the **Metric filters** tab.
3. **Create Filter:**
    - **Filter Pattern:** Type "ERROR" or "404".
    - **Metric Details:** Assign a namespace (e.g., MyAppErrors) and a metric name (ErrorCount).
4. **Result:** You can now create a CloudWatch Alarm based on the number of "Errors" appearing in your text logs.





Observation Table

Task	Observed Event/Metric	Status (Pass/Fail)
CloudTrail Audit	RunInstances detected?	Pass
Metric Tracking	CPUUtilization visible?	Pass
Alarm Trigger	Notification received?	Pass
Dashboard	All widgets rendering?	Pass

Conclusion

By completing this lab, you have established a robust monitoring framework. You can now audit who did what via **CloudTrail**, monitor performance via **CloudWatch Metrics**, automate notifications via **Alarms**, and visualize your entire environment using **Dashboards**.